

Práctica 2: Indexador

Análisis de Implementación y Decisiones de Diseño

1. Introducción

El objetivo de esta práctica es construir la clase `IndexadorHash`, que recibe una colección de documentos de texto y genera las estructuras necesarias para la fase posterior de búsqueda. El indexador aplica la clase `Tokenizador` desarrollada en la Práctica 1 filtra palabras de parada, aplica stemming opcional y construye un índice invertido almacenado en tablas hash (`unordered_map` y `unordered_set` de la STL).

Quiero mencionar no haber sido aún evaluado de la parte de del tokenizador, esto significa que la complejidad espacial de mi práctica seguramente se vea afectada por la de la pasada.

Hipótesis de partida:

- Las operaciones de búsqueda (`find`, `Devuelve`, `Existe`) se ejecutarán mucho más frecuentemente que las inserciones en la fase de recuperación; por ello conviene minimizar su coste.
- El corpus de evaluación puede contener millones de términos únicos, lo que hace inviables estructuras con coste de búsqueda $O(\log n)$ en favor de $O(1)$ amortizado.
- Los documentos pueden reindexarse si se modifican, por lo que el borrado debe ser correcto sin reconstruir todo el índice.

2. Análisis de la Solución Implementada

La solución se centra en la creación de un Índice Invertido eficiente que permite mapear términos con los documentos donde aparecen. Para esto uso las estructuras stl del enunciado para poder tener lecturas y escrituras rápidas

La Estructura Fecha y la Reindexación

Para gestionar el tiempo, he definido la estructura `Fecha` en `indexadorInformacion.h`.

- Representación Interna: Utiliza `time_t` tiempo, que almacena el valor numérico del tiempo de Unix. Esto hace que las comparaciones sean rápidas (comparación de enteros).
- Lógica de Comparación: Incluye el método `esMasReciente(const Fecha& o)`, que devuelve un booleano tras comparar los campos tiempo.

- **Uso en el Algoritmo:** Dentro del método Indexar, utilizo la función stat del sistema para extraer la fecha de modificación del archivo (st_mtime). Si el documento ya existía, el algoritmo invoca esMasReciente. Si la fecha del disco es mayor que la guardada en InfDoc, se activa el proceso de borrado y nueva indexación.

Estructuras de Datos de Soporte

Además de la fecha, el sistema se apoya en una jerarquía de tablas Hash para maximizar la eficiencia:

- **indice:** Un unordered_map<string, InformacionTermino> que actúa como el diccionario global de la colección.
- **indiceDocs:** Un unordered_map<string, InfDoc> que mapea la ruta del archivo con su información estadística y su idDoc único.
- **I_docs:** Situado dentro de cada término, es otro unordered_map<int, InfTermDoc>. Esta estructura de "doble hash" permite que la consulta de un término en un documento concreto sea de coste $O(1)$.
- **stopWords:** Un unordered_set<string> que garantiza que el filtrado de palabras vacías no penalice el tiempo de ejecución.

Flujo del Algoritmo de Indexación

El proceso de indexación sigue un flujo de "comprobación antes de ejecución":

1. **Captura de Metadatos:** El algoritmo obtiene el tamaño y la fecha del archivo mediante llamadas al sistema.
2. **Decisión de Reindexación:**
 - Si el archivo es nuevo, se le asigna el siguiente ID disponible.
 - Si el archivo ya está en indiceDocs pero fechaActual.esMasReciente(fechaGuardada) es cierto, se invoca BorraDoc para limpiar los términos antiguos sin perder el ID del documento.
3. **Procesamiento de Contenido:** Se realiza la lectura, tokenización, filtrado de stop-words y aplicación del stemmer para normalizar las raíces.
4. **Almacenamiento:** Se actualizan las frecuencias (ft y ftc) y, si está habilitado, se inserta la posición en la list<int> posTerm.

3. Pruebas que no han salido

Lectura en bloque del fichero (Cargar todo en memoria)

Intenté optimizar el método `IndexarDocumento`. En lugar de leer el documento línea a línea con `getline()`, hice una versión que leía todo el contenido del fichero de golpe y lo guardaba en un único string gigante, para pasárselo al Tokenizador en una sola llamada, usando la función que implementamos ya de `tokenizarDocumento`. Al probarlo con la colección entera, la complejidad espacial se creció mucho. Cargar ficheros enteros en un string hacía que el consumo de RAM tuviera picos muy grandes. Volví a la lectura línea a línea porque, aunque hace más operaciones I/O, mantiene el consumo de memoria muy predecible. Aunque estoy seguro que esto se puede optimizar más, sobre todo por la complejidad espacial del programa.

Uso de `std::map` para mantener el índice ordenado

Al principio quise probar con `std::map` en lugar de `std::unordered_map` para el índice principal. La ventaja de esta prueba era que al llamar al método `ListarTerminos()`, los terminos ya estaban ordenados alfabéticamente, pero luego en cuanto hice unas pocas pruebas vi que era mucho más ineficiente que `unordered_map` y tuve que volver a la propuesta del enunciado. Ya que el coste temporal subía mucho.

4. Análisis de las mejoras realizadas en la práctica

Delegación de I/O de Directorios al Sistema Operativo:

Para el método `IndexarDirectorio`, la solución clásica en C++ implica usar librerías de tratamiento de ficheros para hacer un recorrido recursivo por las carpetas. Mi mejora ha sido delegar este trabajo directamente al kernel de Linux mediante la llamada `system("find " + dir + " -type f | sort > .lista_tmp")`. El sistema operativo realiza la búsqueda recursiva a nivel de sistema de archivos de forma optimizada y me devuelve un fichero de texto plano temporal que el método `Indexar` lee secuencialmente.

Gestión defensiva en memoria:

Si mientras estamos indexando guardamos todo en el `unordered_map`, tenemos el riesgo de que la memoria puede fallar y perder el progreso hasta el momento. Esto lo he arreglado con un `try-catch`, considerando el error `std::bad_alloc`, si la tabla hash se queda sin memoria se podrán cerrar los ficheros de forma segura, emitir el error que toque y devolver `false`, la gracia de este cambio es que lo que ya estaba preparado para indexar no se pierde.

Cambio posTerm de list a vector

En InformacionTermDoc y pregunta los atributos posTerm, si en lugar de declararlos como listas los hago como vectores la complejidad espacial del programa mejora mucho. Por lo que lo he implementado, es un cambio de estructura de datos que no modifica el flujo del programa, pero lo hace más ligero.

5. Análisis de la complejidad

Para medir esto, defino las variables clave:

- N: Número total de palabras a procesar en un documento.
- V: Tamaño del vocabulario (total de palabras únicas en el índice).
- D: Número de documentos indexados.

Complejidad Temporal

- Indexación (IndexarDocumento): $O(N)$ El bucle principal lee el archivo línea a línea y palabra por palabra (N iteraciones). Dentro del bucle, buscar en las stop-words, aplicar el stemmer y hacer el insert en el unordered_map cuesta $O(1)$ gracias al hashing. Por tanto, el tiempo de indexación crece de forma estrictamente lineal según el tamaño del documento.
- Borrado (BorraDoc): $O(V)$ Para borrar un documento, el programa ejecuta un while que recorre todo el mapa de términos (indice.begin() a indice.end()) buscando si el documento aparece en cada uno. Si el vocabulario V es enorme, el borrado será lento, independientemente de si el documento borrado era grande o pequeño.
- Consultas (Existe, Devuelve): $O(1)$ Al usar tablas Hash, buscar cualquier palabra es acceso directo. No hay bucles dependientes del tamaño del problema.

Complejidad Espacial

- Costo de Memoria: $O(V+D+N)$ El consumo de RAM depende de tres factores:
 1. $O(V)$: El número de claves (palabras únicas) guardadas en el índice principal.
 2. $O(D)$: Los metadatos guardados en indiceDocs por cada fichero.
 3. $O(N)$: El almacenamiento de las posiciones (posTerm). Esta es la parte más pesada. Como cada aparición de una palabra crea un nodo en

una `std::list`, la memoria requerida crece linealmente con la cantidad total de texto ingerido.